

# Jogo da Velha

## com Padrão MVVM no Android

Guia passo a passo · Fácil de seguir

Este guia foi feito para te ajudar a entender e montar o projeto de forma simples. Siga cada passo com calma. Não precisa decorar tudo — use como consulta!

## 1. O que é MVVM? (entenda em 1 minuto)

MVVM significa **Model – View – ViewModel**. É uma forma de organizar o código separando as responsabilidades em três partes:

| Parte     | O que faz                                    | Exemplo no Jogo da Velha                                 |
|-----------|----------------------------------------------|----------------------------------------------------------|
| Model     | Guarda os dados e regras do jogo             | Tabuleiro (matriz 3x3), vez de quem joga, se ganhou      |
| ViewModel | Faz a ponte: processa ações e atualiza dados | Recebe clique na célula, verifica vencedor, avisa a View |
| View      | Mostra a tela para o usuário                 | Botões do tabuleiro, texto de status, botão Reiniciar    |

■ **Dica:** Pensa assim: a View é o que você VÊ. O Model é o que o jogo SABE. O ViewModel é o que o jogo FAZ quando você clica.

## 2. Estrutura de Pastas do Projeto

O projeto Android segue esta organização de arquivos:

| Pasta / Arquivo              | Para que serve                                           |
|------------------------------|----------------------------------------------------------|
| model/GameModel.kt           | Lógica do jogo: tabuleiro, verificar vencedor, reiniciar |
| viewmodel/GameViewModel.kt   | Controla o estado e expõe LiveData para a View           |
| view/MainActivity.kt         | Activity principal — observa LiveData e atualiza a UI    |
| res/layout/activity_main.xml | Layout XML com os 9 botões do tabuleiro                  |

## 3. Model — As Regras do Jogo

O **Model** é a parte mais importante para a nota! Ele contém a lógica pura do jogo, sem saber nada sobre telas ou botões.

### 3.1 O que o Model precisa ter:

#### Responsabilidades do GameModel

- board: Array 3x3 com os valores (' ', 'X', 'O')
- currentPlayer: letra do jogador atual ('X' ou 'O')
- makeMove(row, col): registra a jogada
- checkWinner(): verifica se alguém ganhou
- isDraw(): verifica empate
- reset(): reinicia o tabuleiro

### 3.2 Código básico do Model:

```
class GameModel {
    val board = Array(3) { CharArray(3) { ' ' } }
    var currentPlayer = 'X'
    private set

    fun makeMove(row: Int, col: Int): Boolean {
        if (board[row][col] != ' ') return false // célula ocupada
        board[row][col] = currentPlayer
        return true
    }

    fun checkWinner(): Char? {
        // Verifica linhas
        for (i in 0..2) {
            if (board[i][0] != ' ' &&
                board[i][0] == board[i][1] &&
                board[i][1] == board[i][2])
                return board[i][0]
        }
        // Verifica colunas
        for (j in 0..2) {
            if (board[0][j] != ' ' &&
                board[0][j] == board[1][j] &&
                board[1][j] == board[2][j])
                return board[0][j]
        }
        // Verifica diagonais
        if (board[0][0] != ' ' &&
            board[0][0] == board[1][1] &&
            board[1][1] == board[2][2])
            return board[0][0]
        if (board[0][2] != ' ' &&
            board[0][2] == board[1][1] &&
            board[1][1] == board[2][0])
            return board[0][2]
        return null // ninguém ganhou ainda
    }
}
```

```

fun isDraw(): Boolean {
    return board.all { row -> row.all { it != ' ' } }
}

fun switchPlayer() {
    currentPlayer = if (currentPlayer == 'X') 'O' else 'X'
}

fun reset() {
    for (i in 0..2) for (j in 0..2) board[i][j] = ' '
    currentPlayer = 'X'
}
}

```

## 4. ViewModel — O Controlador do Jogo

O **ViewModel** usa a classe **AndroidViewModel** ou **ViewModel** do Jetpack. Ele expõe **LiveData** — dados que a View observa automaticamente.

### 4.1 O que o ViewModel precisa ter:

#### Responsabilidades do GameViewModel

- Instância do GameModel (não cria ela na View!)
- LiveData para status do jogo (ex: 'Vez do X', 'X Ganhou!')
- LiveData para o estado do tabuleiro
- Função onCellClicked(row, col): chamada pela View ao clicar
- Função onRestart(): reinicia o jogo

### 4.2 Código básico do ViewModel:

```

class GameViewModel : ViewModel() {
    private val model = GameModel()

    // MutableLiveData é privado (só o ViewModel muda)
    private val _status = MutableLiveData("Vez do X")
    val status: LiveData = _status

    private val _board = MutableLiveData<>()
    val board: LiveData<> = _board

    private val _gameOver = MutableLiveData(false)
    val gameOver: LiveData = _gameOver

    init {
        _board.value = model.board
    }
}

```

```

    }

    fun onCellClicked(row: Int, col: Int) {
        if (_gameOver.value == true) return

        val moved = model.makeMove(row, col)
        if (!moved) return // célula já ocupada

        _board.value = model.board // atualiza tabuleiro

        val winner = model.checkWinner()
        when {
            winner != null -> {
                _status.value = "■ Jogador $winner ganhou!"
                _gameOver.value = true
            }
            model.isDraw() -> {
                _status.value = "■ Empate!"
                _gameOver.value = true
            }
            else -> {
                model.switchPlayer()
                _status.value = "Vez do ${model.currentPlayer}"
            }
        }
    }

    fun onRestart() {
        model.reset()
        _board.value = model.board
        _status.value = "Vez do X"
        _gameOver.value = false
    }
}

```

## 5. View — A Tela que o Usuário Vê

A **View** (MainActivity) observa o LiveData do ViewModel e atualiza os botões conforme o estado do jogo muda. Ela **nunca** toma decisões de lógica!

### 5.1 Layout XML — Os 9 botões do tabuleiro

```

        android:rowCount="3"
        android:columnCount="3">

```

## 5.2 MainActivity — Observar e Reagir

```
class MainActivity : AppCompatActivity() {
    private lateinit var viewModel: GameViewModel
    private lateinit var buttons: Array<

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        viewModel = ViewModelProvider(this)[GameViewModel::class.java]

        // Mapeia os botões do layout
        buttons = arrayOf(
            arrayOf(btn00, btn01, btn02),
            arrayOf(btn10, btn11, btn12),
            arrayOf(btn20, btn21, btn22)
        )

        // Configura cliques
        for (i in 0..2) for (j in 0..2) {
            buttons[i][j].setOnClickListener {
                viewModel.onCellClicked(i, j)
            }
        }

        btnRestart.setOnClickListener { viewModel.onRestart() }

        // Observa o status (texto de quem joga / quem ganhou)
        viewModel.status.observe(this) { msg ->
            tvStatus.text = msg
        }

        // Observa o tabuleiro e atualiza botões
        viewModel.board.observe(this) { board ->
            for (i in 0..2) for (j in 0..2) {
                buttons[i][j].text = board[i][j].toString()
            }
        }
    }
}
```

## 6. Fluxo Completo — O que acontece ao clicar?

| Quem age       | O que acontece                                                       |
|----------------|----------------------------------------------------------------------|
| 1■■■ Usuário   | Toca num botão do tabuleiro na tela                                  |
| 2■■■ View      | Chama <code>viewModel.onCellClicked(i, j)</code>                     |
| 3■■■ ViewModel | Chama <code>model.makeMove(i, j)</code>                              |
| 4■■■ Model     | Registra a jogada no <code>board[i][j]</code>                        |
| 5■■■ ViewModel | Chama <code>model.checkWinner()</code> e <code>model.isDraw()</code> |
| 6■■■ ViewModel | Atualiza <code>_status</code> e <code>_board</code> (LiveData)       |
| 7■■■ View      | LiveData dispara: atualiza botões e texto de status                  |

■ **Dica:** A View só faz UMA coisa: mostrar o que o ViewModel mandar. Nunca coloca if/else de lógica do jogo na Activity!

## 7. Checklist para a Prova ■

| ■ Item                                 | Por que é importante para a nota       |
|----------------------------------------|----------------------------------------|
| Model separado da View                 | Mostra que entendeu MVVM               |
| ViewModel com LiveData                 | Padrão obrigatório do projeto          |
| View só observa, não decide            | Separação correta de responsabilidades |
| <code>checkWinner()</code> funcionando | Lógica principal do jogo               |
| <code>isDraw()</code> implementado     | Cobre cenário de empate                |
| Botão Reiniciar funcionando            | Reset completo do estado               |
| Status exibido na tela                 | Feedback visual para o usuário         |

■■ **Atenção:** O professor vai olhar principalmente se o código está separado em Model / ViewModel / View. Se as três partes existirem e conversarem pelo LiveData, você já garante a nota mínima (6,0)!

## 8. Erros Comuns — Evite esses!

| ■ Erro                                       | ■ Como corrigir                            |
|----------------------------------------------|--------------------------------------------|
| Lógica do jogo dentro da Activity            | Mova toda lógica para o Model ou ViewModel |
| ViewModel acessa <code>findViewById()</code> | Nunca! ViewModel não conhece Views         |

| ■ Erro                                        | ■ Como corrigir                           |
|-----------------------------------------------|-------------------------------------------|
| Não usar LiveData — chamar método diretamente | Use LiveData.observe() na Activity        |
| Esquecer de chamar switchPlayer()             | Sempre trocar jogador após jogada válida  |
| Não checar se célula está ocupada             | makeMove() deve retornar false se ocupada |

## 9. Resumo Rápido — Cole na sua cabeça!

| Classe        | Herda de            | Responsabilidade principal     | Usa LiveData?         |
|---------------|---------------------|--------------------------------|-----------------------|
| GameModel     | Nada                | Tabuleiro + regras             | Não                   |
| GameViewModel | ViewModel()         | Liga Model ↔ View via LiveData | Sim (MutableLiveData) |
| MainActivity  | AppCompatActivity() | UI + observar LiveData         | Sim (observe)         |

***Boa sorte! ■ Você consegue! Siga este guia passo a passo e o professor vai entender a lógica que você aplicou.***